

Lekce 4: Pole, objekty, JSON a DOM

1. Osnova obsahu

A. Úvod do polí

- **Co jsou pole?**
 - Definice a účel.
 - Uspořádané kolekce prvků.
- **Vytváření polí:**
 - Pomocí literálů polí (`[]`).
 - Pomocí konstruktoru `Array` .
- **Přístup k prvkům pole:**
 - Indexování (začíná od 0).
 - Negativní indexování (nativně není v JavaScriptu podporováno).
- **Běžné metody polí:**
 - `push()` , `pop()` , `shift()` , `unshift()` .
 - `forEach()` , `map()` , `filter()` , `reduce()` .
- **Vícedimenzionální pole:**
 - Vytváření a přístup k vnořeným polím.

Co jsou pole?

Definice:

Pole je uspořádaná seznamová struktura sloužící k uložení více hodnot v jedné proměnné. Každá hodnota v poli se nazývá prvek a každý prvek je dostupný skrze číselný index začínající nulou.

Klíčové vlastnosti:

- **Uspořádanost:** Prvky si zachovávají pořadí, v jakém byly vloženy.
- **Přístup přes index:** První prvek je na indexu 0, druhý na indexu 1 atd.
- **Dynamická velikost:** Pole v JavaScriptu můžou dynamicky růst i zmenšovat se přidáváním a odebíráním prvků.
- **Heterogenní prvky:** Pole mohou obsahovat prvky různého datového typu (číselné hodnoty, řetězce, objekty, apod.).

Využití v testovací automatizaci:

- Ukládání seznamů testovacích vstupů nebo datových sad.

- Iterace přes pole pro spuštění podobných testů s různými vstupy.
- Organizace kolekcí výsledků testů nebo chybových zpráv.

Vytváření polí

Základní vytvoření pole:

```
// Pomocí pole literálu
const fruits = ["Apple", "Banana", "Cherry"];

// Pomocí konstruktoru Array (méně časté, většinou se nedoporučuje)
const numbers = new Array(1, 2, 3, 4);
```

Nejlepší postupy:

- Preferujte literály pole pro jednoduchost a čitelnost.
- Volte popisné názvy proměnných, které reflektují uložená data.

```
// Dobrá praxe
const usernames = ["alice", "bob", "charlie"];

// Vyhněte se vágním názvům
const arr = ["data1", "data2"]; // Není popisné
```

B. Úvod do objektů

- **Co jsou objekty?**
 - Definice a účel.
 - Párování klíč-hodnota pro ukládání dat.
- **Vytváření objektů:**
 - Pomocí literálů objektů (`{ }`).
 - Pomocí konstruktoru `Object` .
- **Přístup k vlastnostem objektu:**
 - Tečková notace.
 - Závorková notace.
- **Běžné metody objektů:**
 - `Object.keys()` , `Object.values()` , `Object.entries()` .
- **Vnořené objekty:**
 - Vytváření objektů uvnitř objektů.
 - Přístup ke vnořeným vlastnostem.

Co jsou objekty?

Definice:

Objekt v JavaScriptu je kolekce párů klíč-hodnota. Klíče jsou obvykle řetězce (nebo symboly), které slouží jako identifikátory hodnot, a hodnoty mohou být jakéhokoli typu – včetně dalších objektů nebo polí.

Klíčové vlastnosti:

- **Struktura klíč-hodnota:** Přístup k datům pomocí klíčů místo číselných indexů.
- **Neuspořádané vlastnosti:** Není zaručené pořadí, v jakém jsou vlastnosti definovány.
- **Referenční typ:** Objekty jsou referenční typy, což znamená, že více proměnných může odkazovat na stejný objekt v paměti.
- **Flexibilní struktura:** Objekty lze za běhu rozšiřovat nebo měnit přidáváním či odebíráním vlastností.

Využití v testovací automatizaci:

- Reprezentace složitých testovacích datových struktur (např. uživatelské profily, konfigurační nastavení).
- Ukládání odpovědí API a snadné získávání konkrétních datových bodů.
- Správa stavových dat během provádění testů.

Vytváření objektů

Základní vytvoření objektu:

```
// Pomocí literálu objektu
const user = {
  name: "Alice",
  email: "alice@example.com",
  role: "admin",
};
```

Nejlepší postupy:

- Používejte literály objektu pro jednoduchost.
- Udržujte názvy vlastností popisné.
- Vyhýbejte se příliš složitým objektům; rozdělte je na menší části pokud je to potřebné.

```
// Dobrá praxe
const product = {
  id: 101,
  name: "Wireless Mouse",
  price: 29.99,
  stock: 100,
};

// Špatná praxe: nejasné názvy vlastností nebo míchání nesouvisejících dat
const data = {
  a: "Nějaká hodnota",
  b: 123,
  c: true,
  user: { name: "Bob" }
};
```

Vnořené objekty

Definice:

Vnořený objekt je objekt, který obsahuje jiný objekt jako jednu ze svých vlastností. To umožňuje hierarchickou datovou strukturu, která reprezentuje složitější vztahy.

Klíčové vlastnosti:

- **Hierarchická data:** Vlastnosti mohou být samy objekty, což povoluje víceúrovňové datové reprezentace.
- **Přístup k hlubším vlastnostem:** Přístup k vnořeným vlastnostem vyžaduje víceúrovňové vyhledávání.
- **Užitečnost ve struktuře dat:** Vnořené objekty jsou vhodné pro modelování reálných entit a jejich atributů (např. objekt uživatele obsahující objekt adresy).

Využití v testovací automatizaci:

- Ukládání vícestupňových konfigurací (např. nastavení prostředí, uživatelské přihlašovací údaje s více atributy).
- Organizace komplexních odpovědí API, kde jeden endpoint vrací vnořená data (například detaily uživatele včetně adresy, platebních metod a preferencí).

Vnořené objekty

Příklad vnořeného objektu:

```
const userProfile = {
  name: "Alice",
  email: "alice@example.com",
  address: {
    street: "123 Main St",
    city: "Townsville",
    zip: "12345"
  }
};

// Přístup k vnořeným vlastnostem:
console.log(userProfile.address.city); // "Townsville"
```

Nejlepší praxe:

- Strukturovat objekty tak, aby odrážely reálné entity.
- Vyhýbat se příliš hlubokému zanoření (kvůli údržbě).
- U rozměrných dat raději rozložit na více objektů nebo pole.

Přístup k vlastnostem objektu

Tečková notace:

```
console.log(user.name); // "Alice"
```

Závorková notace:

```
console.log(user["email"]); // "alice@example.com"
```

Nejlepší praxe:

- Preferujte tečkovou notaci, pokud jsou názvy vlastností známe a jsou platné identifikátory.
- Použijte závorkovou notaci, když jsou názvy vlastností dynamické nebo obsahují speciální znaky.

```
const propertyName = "role";
console.log(user[propertyName]); // "admin"
```

C. Porozumění JSON (JavaScript Object Notation)

- Co je to JSON?

- Definice a účel.
- Lehký formát pro výměnu dat.
- **Struktura JSON:**
 - Objekty a pole v JSON.
 - Datové typy podporované v JSON.
- **Konverze mezi JSON a JavaScript objekty:**
 - `JSON.stringify()` – převod JavaScript objektu na JSON řetězec.
 - `JSON.parse()` – převod JSON řetězce na JavaScript objekt.
- **Využití v testovací automatizaci:**
 - Ukládání a správa testovacích dat.
 - Mockování odpovědí API.

Co je to JSON?

Definice:

JSON (JavaScript Object Notation) je lehký, na jazyku nezávislý formát pro výměnu dat. Využívá podmnožinu syntaxe JavaScriptu k prezentaci datových struktur jako text, což usnadňuje čtení, zápis a přenos.

Klíčové vlastnosti:

- **Textový formát:** JSON se ukládá jako řetězec, což usnadňuje jeho přenos přes síť.
- **Páry klíč-hodnota a pole:** JSON zrcadlí objektové a polní struktury JavaScriptu.
- **Podporované datové typy:** Řetězce, čísla, booleany, null, objekty a pole. Funkce a hodnoty undefined nejsou podporovány.
- **Jazyková nezávislost:** Přestože pochází z JavaScriptu, JSON je široce podporován v mnoha programovacích jazycích.

Využití JSON v testovací automatizaci pro testovací data a API:

- **Fixture data:** Ukládejte testovací vstupy v JSON souborech pro snadnou údržbu a aktualizaci testovacích scénářů.
- **Mockování odpovědí API:** Poskytování předdefinovaných JSON odpovědí pro simulaci chování backendu v testovacím prostředí.
- **Data řízené testy:** Parsování JSON pro dynamické zásobení více testovacích případů, což snižuje duplikaci a úsilí.

Konverze mezi JSON a JavaScript objekty

Převod JavaScript objektu na JSON řetězec (`JSON.stringify()`):

```
const userObj = { name: "Bob", email: "bob@example.com" };
const jsonString = JSON.stringify(userObj);
console.log(jsonString); // '{"name":"Bob","email":"bob@example.com"}'
```

Převod JSON řetězce na JavaScript objekt (JSON.parse()):

```
const parsedObj = JSON.parse(jsonString);
console.log(parsedObj.name); // "Bob"
```

Nejlepší postupy:

- Provádějte validaci a sanitizaci JSON před parsováním (pokud pochází z externího zdroje).
- Při parsování neznámých dat zpracováváte chyby pomocí try/catch.

```
try {
  const safeObj = JSON.parse(incomingJson);
  // Použijte safeObj zde
} catch (error) {
  console.error("Neplatná JSON data:", error);
}
```

D. Nejlepší praxe pro práci s poli, objekty, JSON a DOM

- **Čitelnost a udržitelnost kódu:**
 - Používejte popisné názvy pro pole a objekty.
 - Udržujte JSON struktury jednoduché a konzistentní.
- **Výkonové aspekty:**
 - Vyhněte se zbytečným manipulacím s DOM.
 - Optimalizujte operace s poli u velkých datových sad.
- **Validace dat:**
 - Validujte JSON data před parsováním.
 - Ověřte existenci vlastností objektu před přístupem.

F. Praktické příklady

- **Implementace testovacích dat pomocí polí a objektů:**
 - Vytvoření a správa testovacích datových sad.
 - Iterace přes testovací data pomocí metod pole.
- **Práce s JSON v Cypress testech:**

- Mockování odpovědí API pomocí JSON.
- Parsování a využití JSON dat v rámci testů.

2. Praktické aktivity: Cvičení a návrhy webové funkcionality

A. Práce s poli – cvičení

- Cvičení:

- Vytvořte pole objektů uživatelů, každý obsahuje `name`, `email` a `role`.
- Využijte metody pole jako `map()`, `filter()` a `reduce()` k následujícím operacím:
 - Extrahování všech emailových adres.
 - Filtrování uživatelů podle role (např. "admin").
 - Výpočet celkového počtu uživatelů.

- Příklad:

```
const users = [  
  { name: "Alice", email: "alice@example.com", role: "admin" },  
  { name: "Bob", email: "bob@example.com", role: "user" },  
  { name: "Charlie", email: "charlie@example.com", role: "user" },  
  { name: "Dave", email: "dave@example.com", role: "moderator" },  
];  
  
const emails = users.map(user => user.email);  
const admins = users.filter(user => user.role === "admin");  
const totalUsers = users.reduce((count) => count + 1, 0);  
  
console.log(emails);  
console.log(admins);  
console.log(totalUsers);
```

B. Manipulace s objekty – cvičení

- Cvičení:

- Vytvořte objekt reprezentující produkt s vlastnostmi jako `id`, `name`, `price`, a `stock`.
- Napište funkce pro:
 - Aktualizaci množství na skladě.
 - Aplikaci slevy na cenu.
 - Získání informací o produktu.

- **Příklad:**

```
const product = {
  id: 101,
  name: "Wireless Mouse",
  price: 25.99,
  stock: 100,
};

function updateStock(product, quantity) {
  product.stock += quantity;
  console.log(`Aktualizovaný stav: ${product.stock}`);
}

function applyDiscount(product, percentage) {
  product.price -= product.price * (percentage / 100);
  console.log(`Cena po slevě: ${product.price.toFixed(2)}`);
}

function getProductInfo(product) {
  return `${product.name} (ID: ${product.id}) - ${product.price} | Sklad: ${product.stock}`;
}

updateStock(product, -10);
applyDiscount(product, 10);
console.log(getProductInfo(product));
```

C. Práce s JSON v Cypress testech – cvičení

- **Cvičení:**

- Vytvořte JSON soubor (users.json) obsahující pole objektů uživatelů s vlastnostmi jako name , email a role .
- Napište Cypress test, který:
 - Načte JSON data pomocí cy.fixture() .
 - Projde uživatelská data a provede akce jako vytvoření uživatelských účtů nebo ověření detailů uživatelů.

- **Příklad:**

```
// users.json
[
  { "name": "Alice", "email": "alice@example.com", "role": "admin" },
  { "name": "Bob", "email": "bob@example.com", "role": "user" },
  { "name": "Charlie", "email": "charlie@example.com", "role": "user" }
]

// Cypress Test
describe('Registrace uživatelů', () => {
  beforeEach(() => {
    cy.fixture('users').as('usersData');
  });

  it('Registruje více uživatelů z JSON dat', function () {
    this.usersData.forEach(user => {
      cy.visit('/register');
      cy.get('#username').type(user.name);
      cy.get('#email').type(user.email);
      cy.get('#role').select(user.role);
      cy.get('#submit').click();
      cy.contains('Registrace byla úspěšná!').should('be.visible');
    });
  });
});
```

- **Návrh webové funkcionality:**

- Implementujte registrační formulář, kde:
 - Testeři mohou automatizovat zadávání více registrací uživatelů pomocí dat ze JSON souboru.
 - Ověřte, že každá registrace je úspěšná na základě vstupních dat v JSON.

3. Možné studentské otázky

A. Pole a objekty:

1. Jaký je rozdíl mezi polem a objektem v JavaScriptu?

- **Odpověď:**

Pole je uspořádaná kolekce prvků přístupných přes číselné indexy, vhodná pro ukládání

seznamů položek. Objekt je neuspořádaná kolekce párů klíč-hodnota, ideální pro reprezentaci složitějších datových struktur s pojmenovanými vlastnostmi.

2. Jak lze iterovat přes prvky pole?

- **Odpověď:**

Přes pole můžete iterovat pomocí cyklů (`for` , `while`), metod pole (`forEach()` , `map()` , `filter()` , `reduce()`) nebo novější syntaxe jako je `for...of` .

3. Jaké jsou výhody použití objektů pro ukládání dat v testech?

- **Odpověď:**

Objekty umožňují strukturovanou a popisnou reprezentaci dat, což zjednodušuje správu a přístup ke konkrétním vlastnostem. Zvyšují čitelnost a udržitelnost, obzvlášť pokud pracujete se složitými testovacími daty.

B. JSON:

1. Proč je JSON preferovaný pro výměnu dat ve webových aplikacích?

- **Odpověď:**

JSON je lehký, snadno čitelný a zapisovatelný, a je nativně podporovaný v JavaScriptu. Jeho kompatibilita napříč různými platformami a jazyky z něj činí ideální volbu pro výměnu dat mezi klientem a serverem.

2. Mohu v JSON datech zahrnout funkce?

- **Odpověď:**

Ne, JSON podporuje pouze datové typy jako řetězce, čísla, objekty, pole, booleany a null. Funkce nejsou podporovány a nelze je serializovat v JSON.

C. Nejlepší praxe:

1. Proč je důležité vyhýbat se hlubokému zanoření v řídicích strukturách?

- **Odpověď:**

Hluboké zanoření může ztížit čitelnost, pochopení a údržbu kódu. Zvyšuje složitost a pravděpodobnost chyb. Použití 'guard' vět nebo rozdělení kódu do menších funkcí může napomoci snížení zanoření.

2. Jak se uplatňuje princip DRY při psaní funkcí a řídicích struktur?

- **Odpověď:**

DRY (Don't Repeat Yourself – neopakuj se) vybízí k eliminaci duplicitního kódu tím, že abstraktujete opakující se úkoly do znovupoužitelných funkcí nebo využíváte cykly, čímž zvyšujete udržitelnost a snižujete chybovost kódu.

4. Doporučené doplňkové materiály

A. Oficiální dokumentace a návody:

- JavaScriptová pole:
 - [MDN Arrays](#)
- JavaScriptové objekty:
 - [MDN Objects](#)
- JSON:
 - [MDN JSON](#)

B. Tutoriály a články:

- Pochopení JavaScriptových polí a objektů:
 - [W3Schools JavaScript Arrays](#)
 - [W3Schools JavaScript Objects](#)
- Práce s JSON:
 - [FreeCodeCamp JSON Guide](#)
- Manipulace s DOM:
 - [JavaScript.info DOM Introduction](#)

C. Interaktivní výukové platformy:

- Codecademy:
 - [Learn JavaScript Arrays](#)
 - [Learn JavaScript Objects](#)
- FreeCodeCamp:
 - [JavaScript Data Structures: Arrays](#)
 - [JavaScript Data Structures: Objects](#)
- [JavaScript.info](#):
 - [JavaScript Objects](#)
 - [JSON](#)

D. Video tutoriály:

- Traversy Media:
 - [JavaScript Arrays Tutorial](#)
 - [JavaScript Objects Tutorial](#)
- The Net Ninja:
 - [JavaScript JSON Tutorial](#)

- **Academind:**
 - [JavaScript Arrays and Objects](#)

E. Procvičovací platformy:

- **HackerRank:**
 - [JavaScript Arrays Challenges](#)
 - [JavaScript Objects Challenges](#)
- **LeetCode:**
 - [JavaScript Arrays Problems](#)
 - [JavaScript Objects Problems](#)
- **Exercism:**
 - [JavaScript Track - Arrays](#)
 - [JavaScript Track - Objects](#)

F. Komunita a podpora:

- **Stack Overflow:**
 - [JavaScript Arrays Questions](#)
 - [JavaScript Objects Questions](#)
 - [JSON Questions](#)
- **Reddit:**
 - [r/javascript](#)
 - [r/learnjavascript](#)
- **Discord komunita:**
 - Připojte se na Discord servery zaměřené na JavaScript pro okamžitou pomoc a diskusi.

5. Doporučené rozdělení lekce na 3 hodiny

Hodina 1: Pole a objekty (60 minut)

- **Úvod do polí (25 minut):**
 - Definice, vytvoření, přístup k prvkům.
 - Běžné metody pole s příklady.
- **Praktická aktivita:**
 - Studenti vytvářejí a manipulují s poli různými metodami.
- **Úvod do objektů (25 minut):**
 - Definice, vytvoření, přístup k vlastnostem.

- Běžné metody objektu s příklady.
- **Praktická aktivita:**
 - Studenti vytváří objekty a pracují s nimi pomocí metod.
- **Přestávka (10 minut):**
 - Krátká pauza k odpočinku.

Hodina 2: JSON a DOM (60 minut)

- **Porozumění JSON (50 minut):**
 - Definice, struktura, konverze mezi JSON a objekty.
 - Využití v testovací automatizaci.
- **Praktická aktivita:**
 - Studenti vytváří JSON soubory a procvičují parsování i serializaci JSON dat.
- **Přestávka (10 minut):**
 - Krátká pauza k odpočinku.

Hodina 3: Nejlepší praxe a praktické aplikace (60 minut)

- **Nejlepší praxe pro práci s poli, objekty, JSON a DOM (20 minut):**
 - Čitelnost kódu, výkon, validace dat, bezpečnostní zásady.
 - Znovupoužitelnost a modularita.
- **Praktická aktivita:**
 - Studenti refaktorují existující kód do nejlepší praxe.
- **Praktické příklady (25 minut):**
 - Implementace testovacích dat pomocí polí a objektů.
 - Práce s JSON v Cypress testech.
- **Interaktivní ukázka (10 minut):**
 - Projděte si vizualizaci rozsahu proměnných na webové stránce.
- **Q&A sekce (5 minut):**
 - Odpovědi na dotazy studentů.
 - Vysvětlení nejasností a posílení klíčových konceptů z lekce.

6. Další doporučení

A. Interaktivní demonstrace:

- **Živé kódování:**
 - Ukažte tvorbu a manipulaci s poli a objekty v reálném čase.

- Předved'te parsování a serializaci JSON dat.
- Živě provádějte manipulaci s DOM a obsluhu událostí.
- **Ladění pomocí `console.log` :**
 - Používejte `console.log` ke sledování operací s poli a objekty.
 - Kontrolujte JSON data a změny v DOM v konzoli prohlížeče.

B. Přitažlivé vizuály:

- **Diagramy:**
 - Vizualizujte strukturu polí a objektů.
 - Ilustrujte formát JSON a jeho vztah k JavaScript objektům.
 - Ukažte stromovou strukturu DOM pro vysvětlení hierarchie prvků.
- **Ukázky kódu:**
 - Předkládejte jasné a zřetelné ukázky kódu pro demonstraci pojmů.
- **Vývojové diagramy:**
 - Využijte diagramy průběhu pro znázornění toku dat JSON do Cypress testů nebo vlivu manipulace s DOM na prvky webu.

C. Povzbuzujte účast:

- **Párové programování:**
 - Nechte studenty spolupracovat v párech na úlohách s poli a objekty, čímž podpoříte týmovou práci.
- **Živé ankety a kvízy:**
 - Zařaďte rychlé kvízy pro ověření znalostí a zvýšení zapojení.
 - Využívejte ankety k získání okamžité zpětné vazby ohledně porozumění materiálu.

D. Poskytujte jasné instrukce:

- **Krok za krokem:**
 - Nabídněte detailní instrukce pro každou praktickou aktivitu, aby všichni mohli postupovat.
- **Rady pro řešení problémů:**
 - Předvídejte běžné problémy (např. chyby v syntaxi JSON, výběr elementů v DOM) a nabídněte řešení.

E. Podporujte přátelské prostředí:

- **Povzbuzujte k otázkám:**
 - Vytvářejte otevřené prostředí, kde se studenti nebojí ptát na radu.
- **Nabízejte více příkladů:**

- Uvádějte různé příklady ke každému pojmu, abyste pokryli různé styly učení a posílili pochopení.
- **Recenze od spolužáků:**
 - Podporujte studenty, aby si vzájemně kontrolovali kód, což zvyšuje spolupráci a upevňuje nejlepší praxi.

F. Využijte reálné scénáře:

- **Správa testovacích dat:**
 - Ukažte, jak spravovat složitá testovací data pomocí polí a objektů.
- **Mockování API pomocí JSON:**
 - Předved'te, jak v Cypress testech mockovat API odpovědi pomocí JSON dat.